BAB V
STORED PROCEDURE

A. Tujuan

1. Mahasiswa dapat menjelaskan aspek pemrograman di dalam *database* untuk impementasi logika bisnis dan keamanan akses data.
2. Mahasiswa dapat memahami dan menjelaskan komponen-komponen pembuatan *stored procedure*.
3. Mahasiswa dapat mendemonstrasikan pembuatan *stored procedure* untuk akses data di *database* dengan menerapkan logika bisnis tertentu.
4. Mahasiswa dapat mendemonstrasikan pemanggilan *stored procedure* pada sintak query.

B. Uraian Materi

MySQL memiliki aspek pemrograman yang disebut dengan *database stored program*, sering juga disebut sebagai *stored module* atau *stored routine*. Aspek pemrograman pada MySQL berisi kumpulan instruksi komputer yang memiliki nama, disimpan serta dieksekusi di *database server* (Harrison dan Feuerstein 2006). Aspek pemrograman di MySQL secara garis besar dibagi menjadi tiga jenis, yaitu : 1) *stored procedure,* 2) *store functions*, 3) *Triggers.*

Stored procedure merupakan jenis *stored program* yang paling umum digunakan, di dalamnya merupakan kumpulan instruksi komputer yang dieksekusi berdasarkan pemanggilan dan dapat berisi beberapa parameter masukan dan luaran. *Stored procedure* adalah sebuah objek terkompilasi yang disimpan di dalam basis data. Di dalamnya dapat melibatkan perintah DDL dan DML (Harrison dan Feuerstein 2006). Umumnya suatu *Stored Procedure* hanya berisi suatu kumpulan proses yang tidak menghasilnya *value*, biasanya hanya menampilkan saja. Apabila terdapat nilai yang akan diberikan pada variabel yang dideklarasikan di dalam procedure pada saat *run time*, dapat dilakukan dengan melewatkan parameter ketika eksekusi *stored procedure* tersebut. Selain itu, sebuah *stored procedure* juga dapat memanggil *stored procedure* lainnya. Selain itu, melalui *Stored procedure*, SQL tidak akan melakukan loading seluruh tabel yang ter-relasi, tetapi langsung

melakukan filtering berdasarkan query yang dimaksud sehingga dari sisi performa eksekusi, utilitas jaringan, dan keamanan dapat lebih terjaga.

Selain kumpulan instruksi komputer, *stored procedure* juga dapat berisi kumpulan perintah SQL yang disimpan dan dieksekusi di dalam *server* MySQL (Kofler 2005). Menurut Kruckenberg & Pipes (2005) serta Kofler (2005), terdapat beberapa keuntungan penggunaan *stored prceodure* di dalam *database*, yaitu :

a. *Stored procedure* dapat berisi beberapa perintah *query* yang dieksekusi sebagai satu unit perintah. Hal ini tentu akan mengurangi lalu lintas data pada jaringan antara *client* dan *server*, karena berkuranganya jumlah permintaan akses ke *database*.
b. *Stored procedure* dapat digunakan untuk implementasi logika bisnis ketika terdapat permintaan akses data untuk mengurangi jumlah data yang ditampilkan di sisi *client*.
c. *Stored procedure* menyediakan antarmuka permintaan akses data, sehingga perintah akses data tidak dibuat dalam satu perintah *query*, akan tetapi beberapa perintah *query* dapat dipanggil dalam satu perintah akses data.
d. *Stored procedure* menawarkan mekanisme representasi *routine* akses data dan mempermudah pemeliharaan dan pengembangannya
e. *Stored procedure* dapat mengurangi pengulangan kode program. Seringkali beberapa aplikasi mengakses data yang sama, sehingga memungkinkan ada segmen perintah *query* atau kode program yang identik. Jika kode tersebut dibuat ke dalam *stored procedure*, maka tidak hanya pengulangan kode yang dapat dihindari, tetapi semua yang terpengaruh aplikasi lebih mudah dipelihara. Ketika skema database diubah, seringkali hanya satu *stored procedure* yang perlu diubah, dan bukan kode di setiap aplikasi.
f. *Stored procedure* dapat meningkatkan keamanan basis data. Beberapa industri yang menetapkan aspek data sebagai aspek kritis sebagai contoh perbankan, aplikasi yang berhubungan dengan antar muka pengguna tidak diijinkan mengakses data ke tabel secara langsung, akan tetapi harus melewati *stored procedure*. Dengan demikian, aturan pembatasan serta mekanisme keamanan data dapat diterapkan dan dapat dipantau.

Manfaat penerapan *stored procedure* adalah untuk pengelolaan data karyawan. Sebagai contoh, ketika seorang karyawan baru bergabung di sebuah instansi, maka data

pegawai tersebut harus dimasukkan ke dalam basis data. Data detil karyawan akan disimpan di beberapa table yang sesuai. Oleh karena itu, diperlukan beberapa kali eksekusi untuk perintah *insert* ke tiap-tiap tabel terkait. Eksekusi perintah SQL terkadang dilakukan secara berulang kali untuk memenuhi sebuah *task*. Pada skenario tersebut, beberapa perintah SQL dimungkinkan dieksekusi secara bersamaan sebagai satu unit dalam *stored procedure*. Hal ini akan membantu mengurangi beban pada jalur komunikasi jaringan.

Stored procedure adalah sebuah objek terkompilasi yang disimpan di dalam basis data. Di dalamnya dapat melibatkan perintah DDL dan DML. Umumnya suatu *Stored Procedure* hanya berisi suatu kumpulan proses yang tidak menghasilnya *value*, biasanya hanya menampilkan saja. Apabila terdapat nilai yang akan diberikan pada variabel yang dideklarasikan di dalam procedure pada saat *run time*, dapat dilakukan dengan melewatkan parameter ketika eksekusi *stored procedure* tersebut. Selain itu, sebuah *stored procedure* juga dapat memanggil *stored procedure* lainnya. Selain itu, melalui *Stored procedure*, SQL tidak akan melakukan loading seluruh tabel yang ter-relasi, tetapi langsung melakukan filtering berdasarkan query yang dimaksud sehingga dari sisi performa eksekusi, utilitas jaringan, dan keamanan dapat lebih terjaga.

1. Sintaks *Stored Procedure*

Stored procedure dibuat menggunakan perintah *CREATE PROCEDURE*. Struktur sintaks suatu *stored procedure* ditunjukkan pada gambar 5.1.

```
CREATE
    [DEFINER = { user | CURRENT_USER }]
    PROCEDURE sp_name (proc_parameter[,...])
    [characteristic ...] routine_body
```

Gambar 5.1. Struktur Sintaks *Stored Procedure*

Keterangan :

- sp_name : nama *stored procedure*.
- proc_parameter : parameter input / output dari *stored procedure* tersebut (opsional).
- characteristic : menjelaskan karakteristik dari stored procedure (COMMENT, LANGUAGE SQL, dan lain-lain).
- routine_body : kumpulan perintah pada *stored procedure* tersebut.

Routine body pada *stored procedure* berisi satu atau beberapa perintah SQL yang akan dijalankan ketika *stored procedure* dieksekusi. *Routine body* diawali dengan statemen "BEGIN" dan diakhiri dengan statemen "END". Seperti contoh pada gambar 5.2.

```
customer: BEGIN
<SQL statement>;
<SQL statement>;
END customer
```

Gambar 5.2. Struktur *Routine Body*

- INVOKER / DEFINER. *DEFINER* dispesifikasikan untuk memutuskan *stored procedure* tersebut dijalankan hanya oleh pengguna yang membuat *stored procedure* tersebut (dalam format penulisan user@host). Jika tidak dispesifikasikan, maka user yang melakukan perubahan (CURRENT_USER) adalah pilihan default. Sedangkan INVOKER memberikan hak akses kepada pengguna tertentu untuk dapat mengekseskusi suatu *stored procedure* (Kruckenberg dan Pipes 2005).

Sebagai contoh, akan dibuat *stored procedure* untuk menampilkan NIP, Nama Dosen beserta Kode Mata Kuliah dan Nama Mata Kuliah yang diampu. Tabel yang yang berhubungan adalah tabel dosen untuk menampilkan nama dosen, tabel mata kuliah untuk mendapatkan nama mata kuliah yang diampu, dan tabel jadwal untuk mendapatkan data pengampuan mata kuliah oleh dosen. Sintaks *stored procedure* untuk menangani kasus di atas ditunjukkan pada gambar 5.3.

```
DELIMITER  //
CREATE PROCEDURE sp_dosen_mk()
BEGIN
        SELECT j.Nip,d.Nama_Dosen,j.Kode_MK,mk.Nama_MK
        FROM jadwal j
        JOIN dosen d ON j.Nip = d.Nip
        JOIN matakuliah mk ON j.Kode_MK=mk.Kode_MK;
END//
```

Gambar 5.3. Sintaks *Stored Procedure* Dosen dan Mata Kuliah Diampu

Berdasarkan sintaks *stored procedure* pada gambar 5.3, dapat dilihat dibagian awal sintaks terdapat deklarasi "DELIMITER //". *Delimiter* digunakan untuk mengatasi permasalah penggunaan tanda titik koma (;), MySQL menganggap tanda titik koma (;) sebagai penanda akhir suatu statemen atau perintah. Sehingga, jika tidak terdapat

deklarasi *delimiter* selain tanda titik koma, eksekusi pembuatan *stored procedure* akan berhenti di "tengah jalan", karena menemukan tanda titik koma dan perintah dianggap sudah selesai. Dalam kasus di atas, *delimiter* yang digunakan adalah tanda "//", maka tanda tersebut juga harus dideklarasikan di akhir perintah pembuatan *stored procedure*.

2. Eksekusi *Stored Procedure*

Eksekusi *Stored Procedure* dapat dilakukan dengan menggunakan perintah "CALL nama_stored_procedure (parameter *input* (*jika ada*))". Sebagai contoh, untuk mengesekusi *stored procedure* yang dibuat sebelumnya, yaitu "sp_dosen_mk", maka sintaks eksekusi *stored procedure*-nya adalah sebagai berikut : "*CALL sp_dosen_mk()*". Pada bagian parameter dikosongkan karena *stored procedure* yang dibuat sebelumnya tidak memerlukan parameter masukan atau luaran. Hasil eksekusi *stored procedure* ditunjukkan pada gambar 5.4.

Nip	Nama_Dosen	Kode_MK	Nama_MK
2160898	Noven Indra, S.Kom.,M.Kom	130342217	Struktur Data dan Algoritma
2160887	Khatibul Umam, S.T.,M.Kom.	130342218	Sistem Cerdas
2160898	Noven Indra, S.Kom.,M.Kom	1403040202	Basis Data Lanjut
2160422	Bangun Wijayanto, S.T.,M.Cs.	1403042108	Teori Komputasi
2160577	Anton Sanjaya, S.Kom.,M.Kom.	1403042110	Manajemen Proyek
2160897	Irsyad Rizki, S.Kom.,M.Kom.	1403042111	Sistem Operasi
2160887	Khatibul Umam, S.T.,M.Kom.	1403043119	Komputasi Lunak
2160588	Teguh Wahyono, S.T.,M.T.	1403043122	Kecerdasan Bisnis

Gambar 5.4. Hasil Pemanggilan *Stored Procedure*

3. Perubahan dan Penghapusan *Stored Precodure*

Struktur *Stored procodeure* dapat diubah menggunakan statemen "ALTER". Sintaks untuk mengubah *stored procedure* menggunakan perintah "ALTER" ditunjukkan pada gambar 5.5.

```
ALTER PROCEDURE [<database>.]<procedure name> <characteristics>
```

Gambar 5.5. Struktur Sintak Mengubah *Stored Procedure*

Sebagai contoh, untuk mengubah hak akses *stored procedure* dengan menambahkan klausa "*SQL SECURITY INVOKER / DEFINER*" dan klausa "*COMMENT*". Contoh sintaks untuk mengubah karakteristik atau struktur *stored procedure* adalah sebagai berikut :

```
ALTER PROCEDURE get_customers SQL SECURITY INVOKER COMMENT
'show all customers';
```

Sedangkan bagian *routine_body* pada sebuah *stored procedure* tidak dapat dimodifikasi menggunakan perintah ALTER PROCEDURE. Untuk mengubah *routine_body* pada sebuah *stored procedure* digunakan perintah "DROP procedure IF EXIST", kemudian deklarasikan kembali perintah pembuatan *stored procedure* (Kruckenberg dan Pipes 2005). Sebagai contoh untuk mengubah *stored procedure* "sp_dosen_mk ". Sintaks untuk mengubah *stored procedure* "sp_dosen_mk" ditunjukkan pada gambar 5.6.

```
DELIMITER //
DROP PROCEDURE IF EXISTS `db_akademik`.`sp_dosen_mk`;

CREATE DEFINER=`root`@`localhost` PROCEDURE `sp_dosen_mk`()
BEGIN
        SELECT j.Nip,d.Nama_Dosen,j.Kode_MK,mk.Nama_MK,J.Hari
        FROM jadwal j
        JOIN dosen d ON j.Nip = d.Nip
        JOIN matakuliah mk ON j.Kode_MK=mk.Kode_MK;
END//
```

Gambar 5.6. Mengubah *Stored Procedure*

4. Menggunakan Parameter pada *Stored Procedure*

Stored procedure dapat berisi parameter, yaitu nilai yang akan dilewatkan pada *stored procedure* secara *run time* saat *stored procedure* tersebut dieksekusi. Parameter di dalam *stored procedure* dapat didefinisikan sebagai "IN", "OUT', atau "INOUT". Parameter yang didefinisikan sebagai "IN" berarti digunakan sebagai parameter masukan yang akan dilewatkan di suatu *stored procedure*, parameter "OUT" berarti *stored procedure* dapat menetapkan nilai luaran yang akan dikembalikan dari *stored procedure* ke "pemanggil" *stored procedure* (DuBois et.al 20025). Sedangkan parameter "INOUT" adalah parameter yang dapat digunakan sebagai nilai masukan pada *stored procedure* dan nilai yang disimpan pada nilai masukan dapat diubah oleh *stored procedure* berdasarkan hasil pengolahan pada *stored procedure* (Harrison dan Feuerstein 2006).

Setiap parameter memiliki nama, tipe data, variabel dimana nilai tersebut akan disimpan, dan nilai awal. Sebagai contoh *stored procedure* untuk menampilkan mata kuliah beserta dosen pengampu pada hari tertentu. Parameter masukan yang akan diproses di dalam *stored procedure* adalah nama hari. Nilai nama hari disimpan di dalam parameter "IN" dengan nama "hari_kuliah". Sintak *stored procedure* tersebut ditunjukkan pada gambar 5.7.

```
DELIMITER//
CREATE PROCEDURE sp_mk_hari
(
        IN hari_kuliah VARCHAR(12)
)
BEGIN
        SELECT j.Nip,d.Nama_Dosen,j.Kode_MK,mk.Nama_MK,j.Hari
        FROM jadwal j
        JOIN dosen d ON j.Nip = d.Nip
        JOIN matakuliah mk ON j.Kode_MK=mk.Kode_MK
        WHERE j.Hari=hari_kuliah;
END//
```

Gambar 5.7. *Stored Procedure* dengan Nilai Masukan

Eksekusi *stored procedure* pada gambar 5.7. dapat dilakukan dengan sintaks berikut :

CALL sp_mk_hari('Selasa');

Nilai masukan yang disimpan di dalam parameter "hari_kuliah" adalah "Selasa". Kemudian perintah query pada *stored procedure* akan mencari kode mata kuliah pada tabel jadwal yang memiliki nilai "Selasa" pada kolom "Hari" melalui sintaks "WHERE j.Hari=hari_kuliah". Hasil eksekusi sintaks *stored procedure* pada gambar 5.7. ditunjukkan pada gambar 5.8.

Nip	Nama_Dosen	Kode_MK	Nama_MK	Hari
2160898	Noven Indra, S.Kom.,M.Kom	130342217	Struktur Data dan Algoritma	Selasa
2160897	Irsyad Rizki, S.Kom.,M.Kom.	1403042111	Sistem Operasi	Selasa
2160897	Irsyad Rizki, S.Kom.,M.Kom.	1403042110	Manajemen Proyek	Selasa
2160897	Irsyad Rizki, S.Kom.,M.Kom.	1403043122	Kecerdasan Bisnis	Selasa

Gambar 5.8. Hasil Eksekusi *Stored Procedure* dengan Nilai masukan

Parameter masukan pada *stored procedure* dapat digunakan untuk menambahkan data, mengubah, atau menghapus data pada suatu tabel. Sebagai contoh *stored procedure* pada gambar 5.9. digunakan untuk menambahkan *record* baru di tabel dosen.

```
DELIMITER //
CREATE PROCEDURE tambah_dosen (
        IN NIP_Dosen VARCHAR(12),
        IN NAMA_DOSEN VARCHAR(50)
)

BEGIN
        INSERT INTO dosen (Nip,Nama_Dosen)
        VALUES (NIP_Dosen,NAMA_DOSEN);
END//
```

Gambar 5.9. *Stored Procedure* untuk Menambah Data

Berdasarkan sintaks *stored procedure* pada gambar 5.9, parameter masukan yang akan dilewatkan adalah "NIP_Dosen" dan "NAMA_DOSEN". Penggunaan *stored procedure* untuk menambahkan data pada tabel dosen dapat dilakukan dengan memanggil *stored procedure* beserta parameter masukkannya, yaitu NIP,dan NAMA_DOSEN. Sintaks pemanggilan *stored procedure* adalah berikut :

call tambah_dosen ('2160587','Suyanto, S.T.,M.T.')

Nilai parameter masukan yang akan diproses di dalam *stored procedure* adalah "NIP_Dosen" bernilai "2160587", sedangkan nilai parameter "NAMA_DOSEN" adalah "Suyanto, S.T., M.T.". Hasil eksekusi *stored procedure* pada gambar 5.9. ditunjukkan pada gambar 5.10.

Nip	Nama_Dosen
2160422	Bangun Wijayanto, S.T.,M.Cs.
2160577	Anton Sanjaya, S.Kom.,M.Kom.
2160587	Suyanto, S.T.,M.T.
2160588	Teguh Wahyono, S.T.,M.T.
2160887	Khatibul Umam, S.T.,M.Kom.
2160897	Irsyad Rizki, S.Kom.,M.Kom.
2160898	Noven Indra, S.Kom.,M.Kom
2160899	Jaya Suprana, S.H,M.Hum
2160900	Sumarwadi, S.Si.,P.hd
2160910	Sujarwadi, S.Si.,P.hd

Gambar 5.10. *Stored Procedure* untuk Menambah Data

5. Mengembalikan nilai pada *Stored Procedure*

Sama halnya dengan pemberian nilai masukan pada *stored procedure* saat *run time*, *stored procedure* juga dapat digunakan untuk mengembalikan nilai sebagai luaran dari *stored procedure*. Nilai dapat dikembalikan melalui parameter OUT. Untuk

menentukan parameter sebagai sebuah luaran, dapat digunakan keyword OUT pada bagian deklarasi parameter. Sebagai contoh, *stored procedure* berikut digunakan untuk menghitung jumlah mata kuliah yang diikuti tiap mahasiswa. Parameter luaran yang dideklarasikan adalah NAMA untuk menyimpan nama mahasisawa dan jumlah_mk untuk menyimpan jumlah mata kuliah yang diikuti merupakan parameter luaran. Sintaks s*tored procedure* tersebut ditunjukkan pada gambar 5.11.

```
DELIMITER//
CREATE PROCEDURE sp_mahasiswa_jml_mk
(
                IN NIM_mhs VARCHAR (12),
                OUT NIM VARCHAR (12),
                OUT NAMA VARCHAR (25),
                OUT jumlah_mk SMALLINT
)
BEGIN
                SELECT k.NIM,m.Nama_Mhs,COUNT(k.Kode_MK) INTO NIM,NAMA,jumlah_mk
                FROM krs k
                JOIN mahasiswa m ON k.NIM=m.Nim
                WHERE k.NIM=NIM_mhs;
END//
```

Gambar 5.11. *Stored Procedure* dengan Nilai Luaran

Berbeda dengan *stored function*, *stored procedure* tidak dapat mengembalikan sebuah nilai, walaupun ada parameter yang dideklarasikan sebagai parameter luaran. Untuk menampilkan nilai luaran pada *stored procedure* digunakan perintah *"SELECT"* (Kofler 2005). Contoh pemanggilan *stored procedure* yang menggunakan parameter OUT dapat dilakukan menggunakan sintaks seperti ditunjukkan pada gambar 5.12.

```
CALL sp_mahasiswa_jml_mk('100304001',@NIM,@NAMA,@jumlah_mk);
SELECT @NIM as NIM,@NAMA as Nama,@jumlah_mk AS jumlah_MK;
```

Gambar 5.12. Menampilkan Nilai Luaran pada *Stored Procedure*

Berdasarkan gambar 5.12. terlihat pemanggilan menggunakan perintah *"SELECT"* untuk menampilkan nilai luaran yang disimpan pada parameter "NAMA" dan "jumlah_mk". Hasil eksekusi pemanggilan *stored procedure* tersebut ditunjukkan pada gambar 5.13.

NIM	Nama	jumlah_MK
100304001	DIANA BUDI	5

Gambar 5.13. Hasil Pemanggilan Nilai Luaran *Stored Procedure*

C. Rangkuman

1. *Stored procedure* merupakan jenis *stored program* yang paling umum digunakan, di dalamnya merupakan kumpulan instruksi komputer yang dieksekusi berdasarkan pemanggilan dan dapat berisi beberapa parameter masukan dan luaran. Selain kumpulan instruksi komputer, *stored procedure* juga dapat berisi kumpulan perintah SQL yang disimpan dan dieksekusi di dalam *server* MySQL.
2. Penggunaan *stored procedure* memiliki beberapa keuntungan, yaitu : 1) penerapan logika bisnis dalam akses data, 2) mengurangi lalu lintas data pada jaringan antara *client* dan *server*, 3) menyediakan antarmuka permintaan akses data, 4) mempermudah pemeliharaan dan pengembangan perintah SQL, 5) mengurangi pengulangan kode program, dan 6) meningkatkan keamanan basis data.
3. *Stored procedure* dapat memiliki parameter masukan dan luaran yang diproses di dalam *routine body stored procedure*.
4. Perintah *"SELECT"* digunakan untuk menampilkan nilai parameter luaran di dalam *stored procedure*.

D. Tugas/Latihan

1. Buatlah *stored procedure* dengan menggunakan *subquery* untuk menambahkan ecord pada table jadwal dengan nama dosen 'Irsyad Rizki, S.Kom.,M.Kom.', nama mata kuliah 'Manajemen Proyek', Hari 'Selasa', Ruang "Lab. RPL", dan Jam "13.00-15.00" !
2. Buatlah *stored procedure* untuk menampilkan NIM, nama mahasiswa, dan IP Semester tiap tiap mahasiswa !
3. Buatlah *stored procedure* untuk menampilkan kode mata kuliah, nama mata kuliah, nama dosen pengampu, dan jumlah mahasiswa tiap yang mengikuti tiap mata kuliah !
4. Buatlah *stored procedure* untuk mengubah (*update*) tabel jadwal untuk kode MK ="1403043119"dengan ketentuan sebagai berikut :
   - Pengampu dengan Nip ="2160887" menjadi "2160587"
   - Hari ="Rabu" menjadi "Jum'at"
   - Ruang ="Lab. Cerdas" menjadi "Lab. RPL"

E. Daftar Pustaka

1. DuBois, P., Hinz, S., Pedersen, C., 2005, *MySQL 5.0 Certification Study Guide*, USA : MySQL Press.
2. Harrison, G. & Feuerstein, S., 2006, *MySQL Stored Procedure Programming*, USA : O'Reilly Media.
3. Kofler, M., 2005, *The Definitive Guide to MySQL 5*, USA : Apress.
4. Kruckenberg, M., dan Pipes, J., 2005, *Pro MySQL*, USA : Apress.